

Jetson Nano 기반 Edge AI Runtime 벤치마크 실험 계획서

멀티 런타임 × 비전 모델 성능·전력·메모리 비교 분석

기간: 2025년 (약 8주) | 플랫폼: NVIDIA Jetson Nano | 분야: Edge AI / 온디바이스 추론

1. 프로젝트 개요

1.1 연구 목적

동일한 비전 AI 모델을 Jetson Nano 온디바이스 환경에서 다양한 런타임 프레임워크(PyTorch, TensorRT, ONNX Runtime, TFLite, ncnn)로 실행하고, 각 런타임의 추론 속도(Latency), 메모리 사용량, 전력 소비를 정밀 측정·비교한다. 더 나아가 모델 내부 레이어 수준의 세밀한 분석을 통해 런타임별 최적화 특성과 에너지 효율을 규명하고, 이를 탄소 배출량으로 환산하여 엣지 AI 시스템 설계에 대한 실증적 가이드라인을 제시한다.

1.2 핵심 연구 질문

- 동일 모델에서 런타임 프레임워크 선택만으로 추론 속도가 얼마나 달라지는가?
- TensorRT INT8 양자화는 정확도 손실 없이 에너지 효율을 얼마나 개선하는가?
- 모델 크기(파라미터 수)와 전력 소비는 어떤 비선형 관계를 가지는가?
- Jetson Nano의 5W / 10W 전력 모드 전환이 실제 추론 성능에 미치는 영향은?

1.3 하드웨어 플랫폼

항목	사양	비고
CPU	Quad-core ARM Cortex-A57 @ 1.43GHz	4코어
GPU	128-core Maxwell @ 921MHz	CUDA 지원
RAM	4GB LPDDR4 (CPU·GPU 공유)	스왑 추가 권장
전력 모드	5W MODE / 10W MODE	J48 점퍼로 전환
OS / SDK	Ubuntu 18.04 + JetPack 4.x	CUDA·cuDNN·TRT 포함

2. 실험 설계

2.1 런타임 프레임워크 (4개)

하나의 모델을 아래 4개 런타임에서 순차 실행하여 동일 조건 하에 비교한다.

런타임	실행 방식	Jetson 지원	특징 및 측정 포인트
PyTorch (CPU)	torch.inference_mode	공식 지원	기준값 · register_forward_hook으로 레이어별 분석
PyTorch (CUDA)	model.cuda()	공식 지원	GPU 가속 기준값 · CUDA 이벤트 타이머
TensorRT	FP32 / FP16 / INT8	공식 지원 (JetPack 내장)	TRT Profiler로 레이어별 latency · 최고 최적화
ONNX Runtime	CUDA EP / TRT EP	공식 지원	크로스플랫폼 기준 · 두 가지 EP 비교
ncnn	Vulkan / CPU	커스텀빌드 필요	모발일/엣지 고속 추론 비교 · Vulkan 가속 테스트(C++기반 빌드)
TFLite	GPU delegate	지원	모바일 경량 런타임 비교

2.2 실험 대상 모델 (6개)

경량 → 중량 스펙트럼으로 선정. 하나의 모델을 모든 런타임에서 완전히 돌린 후 다음 모델로 이동한다.

팀 인원이 3명이기에 각 색상별로 1인씩 배정하여 진행한다.

모델	파라미터	태스크	입력 크기	선정 이유
MobileNetV3-Small	~2.5M	이미지 분류	224×224	초경량 기준선
ResNet-50	~25.6M	이미지 분류	224×224	중량 모델 상한선
EfficientNet-B0	~5.3M	이미지 분류	224×224	효율성 최적화 모델
YOLOv8n	~3.2M	객체 탐지	640×640	실시간 탐지 대표 모델
ShuffleNet V2	~2.5M	이미지 분류	224×224	초경량 모델 다양성 확보
SSD-MobileNet V2	~5.0M	객체탐지	300x300	다른 탐지 아키텍처 비교

2.3 측정 지표 및 도구

측정 항목	측정 도구	단위	비고
추론 레이턴시	time.perf_counter / CUDA Event	ms	warm-up 10회, 측정 100회 평균
GPU 메모리	tegrastats / torch.cuda.memory_allocated	MB	CPU·GPU 공유 메모리 주의
전력 소비	tegrastats (INA3221 센서)	mW	5W / 10W 모드 각각 측정
레이어별 latency	PyTorch forward_hook / TRT Profiler	ms/layer	심화 분석 단계
탄소 배출량	codecarbon 라이브러리	gCO ₂ eq	전력(Wh) × 탄소 계수 자동 환산
모델 정확도	ImageNet top-1 accuracy / mAP	%, mAP	런타임별 정확도 차이 검증

2.4 실험 단위: 1 모델 × 전체 런타임 완결 원칙

모든 런타임 실험은 아래 순서로 하나의 모델에 대해 완전히 반복한 후 다음 모델로 넘어간다. 이를 통해 모델 간 비교와 런타임 간 비교를 동시에 수행할 수 있다.

- **Step 1: 모델 준비** — PyTorch 원본 모델 로드 및 정확도 기록¹
- **Step 2: 모델 변환** — ONNX Export → TFLite 변환 / TensorRT Engine 빌드 (FP32·FP16·INT8) / ncnn 모델 변환¹
- **Step 3: 런타임별 추론** — PyTorch CPU → PyTorch CUDA → TensorRT → ONNX Runtime → TFLite → ncnn 순¹
- **Step 4: tegrastats 병렬 로깅** — 각 런타임 추론 중 전력·메모리 실시간 기록¹
- **Step 5: 레이어 분석** — PyTorch hook 및 TRT Profiler로 레이어별 시간 추출¹
- **Step 6: 5W 모드 재측정** — 동일 실험을 5W 전력 모드로 반복¹
- **Step 7: 결과 저장** — CSV / JSON 형태로 구조화 저장¹

3. 주차별 상세 실험 일정 (8주)

1주차 — 환경 구축 및 검증

항목	상세 내용
----	-------

목표	Jetson Nano 완전 세팅 및 모든 프레임워크 설치·동작 확인
작업	JetPack 플래싱 → 스왑 4GB 설정 → PyTorch·TensorRT·ONNX RT·TFLite.ncnn 설치 → CUDA 동작 확인 → tegrastats 로깅 테스트 → SSH 환경 구성
산출물	환경 설정 스크립트(setup.sh) · 동작 확인 스크린샷 · 설치 버전 문서화

2주차 — 측정 파이프라인 개발

항목	상세 내용
목표	자동화된 벤치마크 측정 프레임워크 완성
작업	benchmark.py 작성(warm-up·반복 측정·통계 계산) → tegrastats 파서 개발 → forward_hook 레이어 분석기 → codecarbon 연동 → 결과 CSV 저장 구조 설계
산출물	benchmark.py · tegra_parser.py · layer_analyzer.py · 샘플 측정 결과 CSV

3-4주차 — 모델 1·2 전체 런타임 실험

항목	상세 내용
3주차 4주차	● P1(MobileNetV3-S): 모델 변환 → 5개 런타임 전력·속도·메모리 완전 측정 → 레이어별 분석 → 5W 모드 재측정. ● P2(EfficientNet-B0): 동일 절차 반복. ● P3(ShuffleNetV2): 동일 절차 반복. (총 3개 모델 동시 진행)
산출물	모델 완전 측정 데이터셋 . 1차 비교 차트,

5-6주차 — 모델 3·4 전체 런타임 실험

항목	상세 내용
5주차 6주차	● P1(ResNet-50): 가장 무거운 모델 → OOM 상황 대비 배치 크기 조정 → 5W 모드에서 열 스로틀링 관찰 및 기록. ● P2(YOLOv8n): 객체 탐지 모델 변환 특이사항 처리 → 5개 런타임 측정 → mAP 정확도 검증. ● P3(SSD-MobileNetV2): 객체 탐지 모델 변환 및 측정. (총 3개 모델 동시 진행)
산출물	모델 완전 측정 데이터셋 · 6개 모델 전체 비교 원시 데이터

7주차 — 데이터 분석 및 시각화

항목	상세 내용
분석 내용	런타임별 속도 히트맵 ● · 전력-속도 트레이드오프 산점도 ● · 레이어별 latency 폭포수 차트 · 탄소 배출량 비교 ● · TRT 양자화(FP32→INT8) 정확도-효율 곡선 ● · 5W vs 10W 성능 비교

5. 레이어별 분석 방법

5.1 PyTorch forward_hook (CPU / CUDA)

PyTorch의 `register_forward_hook`을 이용해 각 레이어의 실행 시간과 출력 텐서 크기를 기록한다. 아래는 핵심 측정 코드 구조이다.

```
layer_times = {}  
def hook_fn(module, input, output):  
    layer_times[module.__class__.__name__] = time.perf_counter()  
for layer in model.modules(): layer.register_forward_hook(hook_fn)
```

5.2 TensorRT Profiler

TensorRT의 내장 `IProfiler` 인터페이스를 구현하여 각 레이어의 레이턴시를 측정한다. TRT Engine 실행 시 자동으로 레이어별 ms 단위 시간이 기록된다.

5.3 전력 추이 매핑

`tegrastats`를 100ms 간격으로 별도 프로세스로 실행하면서, 각 레이어 시작·종료 타임스탬프와 전력 로그를 사후 매핑하여 레이어별 전력 소비 추정값을 계산한다.

6. 예상 결과 및 학술적 기여

6.1 예상 핵심 발견

- TensorRT INT8이 PyTorch CPU 대비 추론 속도 5–15배 향상을 보일 것으로 예상
- 5W 모드에서 10W 대비 성능은 30–40% 저하되지만 에너지 효율(성능/와트)은 오히려 향상될 가능성
- 모델 크기와 전력 소비 간 비선형 관계: 파라미터 수 10배 증가 시 전력은 2–3배만 증가
- ONNX Runtime의 TRT EP가 순수 TRT에 근접하지만 변환 오버헤드 존재

6.2 학술적 기여

- Jetson Nano 기반 멀티 런타임 체계적 벤치마크 데이터셋 최초 공개 (GitHub)
- 레이어 수준 전력 소비 추정 방법론 제시
- 탄소 배출량 기반 엣지 AI 모델 선택 가이드라인

7. 리스크 및 대응 방안

리스크	가능성	대응 방안
ResNet-50 OOM (4GB 공유 메모리)	높음	배치 크기 1로 고정, 스왑 파일 4GB 확보
TFLite GPU delegate 미지원 연산	중간	CPU fallback으로 측정 후 별도 표기
열 스로틀링으로 측정값 불안정	중간	측정 전 5분 대기, 쿨링팬 필수, 온도 함께 기록
YOLOv8 TRT 변환 실패	낮음	ONNX 경유 변환, ultralytics 공식 export 활용
ncnn Jetson Vulkan 빌드 실패	중간	CUDA 백엔드로 대체하여 측정